

category: linux-desktop

tags: [x11, xlib, xtest, threading, task-run, remote-control, zcuagent]

severity: critical

created: 2026-07-31

updated: 2026-07-31

project-origin: 6.RemoteControlTool (ZcuAgent)

## "Bỏ qua task chậm, bắt task mới" trên cùng 1 Xlib Display gây domino block, không phải song song thật

### Tình huống gặp phải

Đang làm gì? Tính năng gì? Môi trường nào?

User báo "remote không ổn định" khi dùng tính năng remote control ZcuAgent (điều khiển chuột/bàn phím từ xa qua X11/XTest) trên máy kiosk Ubuntu 22.04 thật (192.168.21.97). SSH vào kiểm tra `journalctl --user -u ipgs-remote-agent.service` phát hiện các đợt burst 66-118 dòng cảnh báo "lệnh inject chuột/bàn phím chạy quá 2000ms" liên tiếp trong vài giây, kèm capture màn hình đang fallback XGetImage (MIT-SHM bị `XShmAttach` từ chối với `BadAccess` — nguyên nhân môi trường, chưa xác định được root cause tận gốc, chỉ biết capture chậm hơn bình thường).

### Triệu chứng / Lỗi

```
warn: ...TcpServer[0]
      Session 192.168.21.15: lệnh inject chuột/bàn phím chạy quá 2000ms (X server có thể
      đang chậm) — tiếp tục xử lý hàng đợi, không chờ lệnh này nữa
[... lặp lại 66-118 lần liên tiếp trong ~5-10 giây ...]
```

Load máy (CPU/RAM) khi kiểm tra đều thấp (load average 0.17-0.6, RAM còn 2.3GB trống)  
— KHÔNG phải do máy quá tải resource. Bug nằm ở tầng code, không phải hạ tầng.

### Nguyên nhân gốc rễ (Root Cause)

`ClientSession.RunInputInjectorLoopAsync` (F25) xử lý hàng đợi lệnh inject như sau  
(bản LỖI):

```
var task = Task.Run(action, ct);
if (await Task.WhenAny(task, Task.Delay(InjectWarnMs, ct)) != task)
{
    _logger.LogWarning("... không chờ lệnh này nữa");
    // KHÔNG await task — đọc luôn lệnh kế tiếp, bắt Task.Run MỚI
}
```

Ý định: nếu 1 lệnh XSync bị X server làm chậm, "bỏ qua" nó để không chặn xử lý Pong.

NHƯNG `MouseInjector/KeyboardInjector` mỗi cái mở \*1 `'Display'` Xlib riêng, dùng CHUNG cho MỌI lệnh `Move/Button/SendKey**` trong suốt vòng đời session. Xlib tự khoá nội bộ MỖI `Display*` bằng 1 mutex (kể cả sau `XInitThreads()` — thread-safe nghĩa là gọi đồng thời từ nhiều thread KHÔNG corrupt protocol, nhưng vẫn SERIALIZE hoàn toàn qua khoá đó, không chạy song song thật).

Hệ quả: khi lệnh #1 đang bị X server làm chậm (đang giữ khoá Display), loop "bỏ qua" nó và bắn `Task.Run` cho lệnh #2 — nhưng lệnh #2 gọi vào CÙNG `Display*` nên chỉ NẪM CHỜ khoá Xlib, không hề chạy. Sau đúng `InjectWarnMs` (2000ms) nữa, lệnh #2 CŨNG bị coi là "quá chậm" (vì nó chưa từng chạy được, chỉ đang xếp hàng chờ khoá) → loop lại bỏ qua nó, bắn `Task.Run` cho lệnh #3... Domino: MỖI lệnh mới vào hàng đợi trong lúc lệnh gốc còn chậm đều tự động tính đủ 2000ms rồi bị coi là "chậm", sinh ra 1 dòng warning MỚI + 1 thread `ThreadPool` MỚI bị block xếp hàng — tạo đúng hiệu ứng burst hàng chục dòng warning liên tiếp quan sát được. "Bỏ qua, không chờ" không hề giải phóng gì cả — nó chỉ làm NỖ SỞ LƯỢNG thread bị khoá.

## Giải pháp

Vẫn giữ nguyên việc LOG cảnh báo sớm ở mốc `InjectWarnMs` (giữ khả năng chẩn đoán X server chậm), nhưng **PHẢI** `'await task' cho xong thật` trước khi đọc lệnh kế tiếp từ hàng đợi — đảm bảo tại một thời điểm chỉ có ĐÚNG 1 lệnh Xlib đang thực thi, loại bỏ hoàn toàn khả năng domino:

```
var task = Task.Run(action, ct);
if (await Task.WhenAny(task, Task.Delay(InjectWarnMs, ct)) != task)
{
    _logger.LogWarning("... vẫn đợi lệnh này xong (không bắn thêm task chồng lên)");
    await task;    // BẮT BUỘC — đây là fix
}
```

File: `IPGS.RemoteControl.ZcuAgent/Net/ClientSession.cs`  
(`RunInputInjectorLoopAsync`).

## Áp dụng lại (How to reuse)

- Khi 1 resource (`Display*`, connection, handle...) chỉ hỗ trợ 1 thao tác tại 1 thời điểm (dù có lock nội bộ tự động serialize), KHÔNG BAO GIỜ dùng pattern "timeout → bỏ qua task cũ, bắn task mới" cho các lệnh gọi VÀO CÙNG resource đó. Lock nội bộ sẽ biến "bỏ qua" thành "xếp hàng vô hạn", timeout sẽ tự lặp lại cho mọi task mới thêm vào trong lúc lock còn bị giữ.
- Dấu hiệu nhận biết pattern lỗi này: số dòng cảnh báo "quá Xms" tăng đột biến theo cấp số nhân/burst lớn (hàng chục-trăm dòng liên tiếp trong vài giây) thay vì lác đác từng dòng — đó là domino, không phải nhiều sự cố độc lập.

- Muốn "bỏ qua thật" (không chờ) một cách an toàn thì phải Hủy được thao tác cũ (VD CancellationToken thật sự ngắt được lời gọi, hoặc mở resource riêng cho mỗi lệnh) — nếu resource dùng chung không huỷ được giữa chừng (như 1 Xlib Display đang trong XSync), "bỏ qua" chỉ là ảo giác.

### Chú ý / Cạm bẫy (Gotchas)

- ⚠ `XInitThreads()` làm Xlib AN TOÀN khi gọi từ nhiều thread (không corrupt), nhưng KHÔNG làm các lời gọi chạy song song — chúng vẫn serialize qua 1 mutex nội bộ mỗi `Display*`. Đừng nhầm "thread-safe" với "chạy song song được".
- ⚠ Log volume tăng đột biến (burst warning) thường là dấu hiệu của vòng lặp/domino logic lỗi, không phải "nhiều sự cố xảy ra cùng lúc" — luôn nghi ngờ code trước khi đổ lỗi cho hạ tầng (X server chậm/máy quá tải) khi thấy pattern burst đều đặn kiểu này, nhất là khi load CPU/RAM đo được lại thấp.
- ⚠ Root cause của việc MIT-SHM bị XShmAttach từ chối (BadAccess) trên máy này VẪN CHƯA xác định được tận gốc (Xorg chạy rootful qua SDDM là bình thường, root thường bypass được permission check SysV shm nên UID mismatch không giải thích được hoàn toàn) — code đã handle graceful fallback đúng, nhưng nếu muốn xoá hẳn lag do XGetImage chậm hơn SHM, cần điều tra thêm ở tầng Xorg/kernel (AppArmor, capability dropping của Xorg wrapper...), KHÔNG thuộc phạm vi bug đã fix ở đây.

### Tham chiếu

- Máy test thật: 192.168.21.97 (kztek), Ubuntu 22.04.5, Xorg qua SDDM autologin.
- File: `IPGS.RemoteControl.ZcuAgent/Net/ClientSession.cs`  
(`RunInputInjectorLoopAsync`,  
đánh dấu F26), `IPGS.RemoteControl.ZcuAgent/Input/MouseInjector.cs`,  
`IPGS.RemoteControl.ZcuAgent/Input/KeyboardInjector.cs`.
- Lesson liên quan: [linux-desktop/gnome-wayland-remote-control-mutter-dbus-pipewire.md](gnome-wayland-remote-control-mutter-dbus-pipewire.md)  
(các bug thật khác của cùng tính năng remote control, verify trên ZCU thật).